

# Atelier — Découvrir MCP en construisant un outil

---

**Public** : développeurs (pairs). **Durée** : ~1 h 30. **Format** : machine + IDE.

## Prérequis participants

---

- Node 20+, le dépôt cloné, `npm install` puis `npm run db:setup` exécutés.
- (Optionnel pour la démo agent) Ollama installé avec `llama3.1`, ou une clé OpenAI.

## Objectifs

---

À la fin, chacun sait expliquer MCP, lire/tester un serveur MCP, **écrire un nouvel outil** et le tester — en autonomie.

## Déroulé minuté

---

### 0–20 min — Théorie (pourquoi MCP ?)

- Le problème Velora : l'information éclatée, le copilote comme réponse.
- MCP = standard ouvert ; les 3 primitives (tools, resources, prompts).
- « Un serveur, plusieurs clients » (Inspector, agent custom, Claude Desktop).
- Sécurité : injection de prompt, *tool poisoning*, principe de moindre privilège.

### 20–35 min — Démonstration

- `npm run inspect` → explorer les outils, appeler `search_products`, `get_order_status` depuis l'Inspector.
- `npm run showcase` → montrer les résultats réels (sans LLM).
- (Si LLM dispo) `npm run agent -- "Où en est la commande VEL-1003 ?"`.

### 35 min–1 h 20 — Coding Kata (pratique)

- Énoncé : ajouter l'outil `get_shipping_estimate` (voir `kata.md`).
- Étapes guidées ; vérification à l'Inspector puis via le test fourni.
- L'animateur circule, débloque, fait verbaliser les choix de schéma.

### 1 h 20–1 h 30 — Débrief & projection

- Comparaison des solutions ; pièges rencontrés (schémas, casse, erreurs).
- Limites et industrialisation : transport HTTP distant, RAG, durcissement sécu.
- Projection : agents multi-serveurs, registre interne d'outils.

## Critères de réussite de l'atelier

---

- Chaque participant a un `get_shipping_estimate` **fonctionnel** visible dans l'Inspector et **vert** au test.
- Chacun sait articuler en une phrase l'intérêt de MCP pour Velora.

# Coding Kata — Ajouter l'outil `get_shipping_estimate`

## Objectif

Ajouter au serveur MCP Velora un outil qui **estime le délai et le tarif de livraison** selon le pays et le poids, en respectant la politique de livraison ( `src/resources/policies/shipping.md` ).

## Spécification

- **Nom** : `get_shipping_estimate`
- **Entrée** : `{ country: string, weightKg: number }`
- **Sortie** (JSON) : `{ zone, etaDays, priceEur }`
- **Règles** :
  - France ( `FR` / « France » ) → zone `FR` , délai « 2 à 3 jours », tarif **4,90 €**.
  - Union européenne (DE, BE, ES, IT, NL, PT, LU...) → zone `EU` , « 4 à 7 jours », **9,90 €**.
  - Hors UE → **erreur** « livraison indisponible ».
  - Surcharge **+2 €** si `weightKg > 2` .

## Étapes guidées

1. Copier `kata-starter/get_shipping_estimate.starter.ts` vers `src/tools/getShippingEstimate.ts` .
2. Implémenter le `handler` (calcul `zone` / `etaDays` / `priceEur` ) en réutilisant `jsonResult` et `errorResult` de `src/tools/util.js` .
3. Enregistrer l'outil dans `src/tools/index.ts` :

```
import { registerGetShippingEstimate } from "../getShippingEstimate.js";
// ...dans registerAllTools(server) :
registerGetShippingEstimate(server);
```

4. Vérifier : `npm run inspect` → l'outil apparaît et répond.
5. Copier `kata-starter/get_shipping_estimate.test.ts` vers `tests/shipping.test.ts` puis lancer `npm test` (doit être **vert**).

## Critères de réussite

- L'outil apparaît dans `tools/list` (Inspector).
- `get_shipping_estimate({country:"FR", weightKg:1})` → 4,90 € , zone `FR` .
- `get_shipping_estimate({country:"DE", weightKg:3})` → 12,90 € (9,90 + 2).
- `get_shipping_estimate({country:"US", weightKg:1})` → **erreur**.

## Solution

---

Une implémentation de référence est fournie dans `kata-starter/get_shipping_estimate.solution.ts` (à ne consulter qu'après essai).

# Squelette du Kata

---

Fichiers fournis :

- `get_shipping_estimate.starter.ts` — **squelette à compléter** (à copier vers `src/tools/getShippingEstimate.ts` ).
- `get_shipping_estimate.test.ts` — **test de validation** (à copier vers `tests/shipping.test.ts` ).
- `get_shipping_estimate.solution.ts` — **solution de référence** (à consulter après avoir essayé).

Suivez les étapes de `../kata.md` .

Ces fichiers ne sont volontairement **ni compilés ni testés** par le projet (hors `src/` et `tests/` ), pour que le dépôt reste « vert » avant le kata.